

Rithika Ravi	P575	B01	"Week 1 — C# Language & Object Foundations".
--------------	------	-----	--

```
Microsoft Visual Studio Debug Console
=== Implicit Conversion (safe widening) ===
int: 42, long: 42, float: 42, double: 42

=== Explicit Casting (possible data loss) ===
double: 3.99 ? int: 3 (decimal lost)

=== 'is' and 'as' (reference types) ===
'is' success: hello
'as' result: hello

=== Convert (throws if invalid) ===
Convert.ToInt32("42") = 42

=== TryParse (safe, no exception) ===
TryParse("abc") success: False
Parsed value: 0

C:\Users\ravi.rithika\source\repos\DayThreeTask\DayThreeTask\bin\Debug\net10.0\DayThreeTask.exe (process 10980) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Day 4 — Modern Syntax + Class Anatomy

Task 1.13 var reassignment compile error; dynamic reassigned string→int→bool with. GetType(); a dynamic-param add method called with ints then strings. ✓ Done when: the compile error is shown, and the runtime types are printed.

Procedure:

Var declared variables can't be reassigned because it gets assigned at compile time. Dynamic declared variables get assigned at runtime. Hence, it can be reassigned anytime. Additionally, we can also use GetType() in-built method for dynamic variables to find the type of value assigned.

Rithika Ravi	P575	B01	"Week 1 — C# Language & Object Foundations".
--------------	------	-----	--

The screenshot shows the Visual Studio Code editor with a C# file named `Program.cs`. The code is as follows:

```
1 using System;
2
3 class Program
4 {
5     0 references
6     static void Main()
7     {
8         // ✓ VAR (compile-time fixed)
9         var x = 10;
10        x = "hello"; //✗ COMPILE ERROR
11
12        // ✓ DYNAMIC (runtime flexible)
13        dynamic d = "hello";
14        Console.WriteLine(d.GetType()); // String
15
16        d = 123;
17        Console.WriteLine(d.GetType()); // Int32
18
19        d = true;
20        Console.WriteLine(d.GetType()); // Boolean
21
22        // ✓ dynamic method
23        Console.WriteLine(Add(5, 10)); // ints
24        Console.WriteLine(Add("Hi ", "Ravi")); // strings
25    }
26
27    2 references
28    static dynamic Add(dynamic a, dynamic b)
29    {
30        return a + b;
31    }
32 }
```

The error message "COMPILE ERROR" is shown next to line 10. The status bar at the bottom indicates "Ln: 9, Ch: 25 SPC CRLF UTF-8 with BOM".

The screenshot shows the Visual Studio Code editor with the `Program.cs` file open. The `Error List` panel is visible at the bottom, showing two errors:

Code	Description	Project	File	Line
ENC00...	Applying source changes while the application is running is not supported by the runtime.	HelloCohort	Program.cs	1
CS0029	Cannot implicitly convert type 'string' to 'int'	HelloCohort	Program.cs	9

The `Error List` panel also shows "2 Errors", "0 Warnings", and "0 of 2 Messages". The status bar at the bottom indicates "Ln: 9, Ch: 14 SPC CRLF UTF-8 with BOM".

Rithika Ravi	P575	B01	"Week 1 — C# Language & Object Foundations".
--------------	------	-----	--

```

1 using System;
2
3 class Program
4 {
5     static void Main()
6     {
7         // VAR (compile-time fixed)
8         var x = 10;
9         //k = "hello"; //X COMPILE ERROR
10
11         // DYNAMIC (runtime flexible)
12         dynamic d = "hello";
13         Console.WriteLine(d.GetType()); // String
14
15         d = 123;
16         Console.WriteLine(d.GetType()); // Int32
17
18         d = true;
19         Console.WriteLine(d.GetType()); // Boolean
20
21         // dynamic method
22         Console.WriteLine(Add(5, 10)); // ints
23         Console.WriteLine(Add("Hi ", "Ravi")); // strings
24     }
25
26     static dynamic Add(dynamic a, dynamic b)
27     {
28         return a + b;
29     }
30 }

```

```

Microsoft Visual Studio Debug:
System.String
System.Int32
System.Boolean
15
Hi Ravi

C:\Users\ravi.rithika\HelloCohort\HelloCohort\bin\Debug\net10.0\HelloCohort.exe (process 22072) exited with code 0 (0x0)
.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|

```

Task 1.14 GetMinMax(int[]) returns named (int Min, int Max); an employee lookup returns (Name, Age, Department) consumed by deconstruction. ✓ Done when: both tuples are returned and deconstructed into separate variables.

Procedure:

Rithika Ravi	P575	B01	"Week 1 — C# Language & Object Foundations".
--------------	------	-----	--

We use tuples to return more than variables as a result. We can pass different values as input types using destructuring. The below methods GetMinMax and GetEmployee are examples of this concept.

```

25 //Task 1.10
26 int[] numbers = { 5, 2, 9, 1 };
27
28 var (min, max) = GetMinMax(numbers);
29 Console.WriteLine($"Min: {min}, Max: {max}");
30
31 var (name, age, dept) = GetEmployee();
32 Console.WriteLine($"{name}, {age}, {dept}");
33
34 }
35
36 1 reference
37 static (int Min, int Max) GetMinMax(int[] nums)
38 {
39     int min = nums[0];
40     int max = nums[0];
41
42     foreach (int n in nums)
43     {
44         if (n < min) min = n;
45         if (n > max) max = n;
46     }
47     return (min, max);
48 }
49
50 1 reference
51 static (string Name, int Age, string Department) GetEmployee()
52 {
53     return ("Ravi", 25, "IT");
54 }
55
56

```

```

Microsoft Visual Studio Debug
System.String
System.Int32
System.Boolean
15
Hi Ravi
Min: 1, Max: 9
Ravi, 25, IT

C:\Users\ravi.rithika\HelloCohort\HelloCohort\bin\Debug\net10.0\HelloCohort.exe (process 17044) exited with code 0 (0x0)
.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

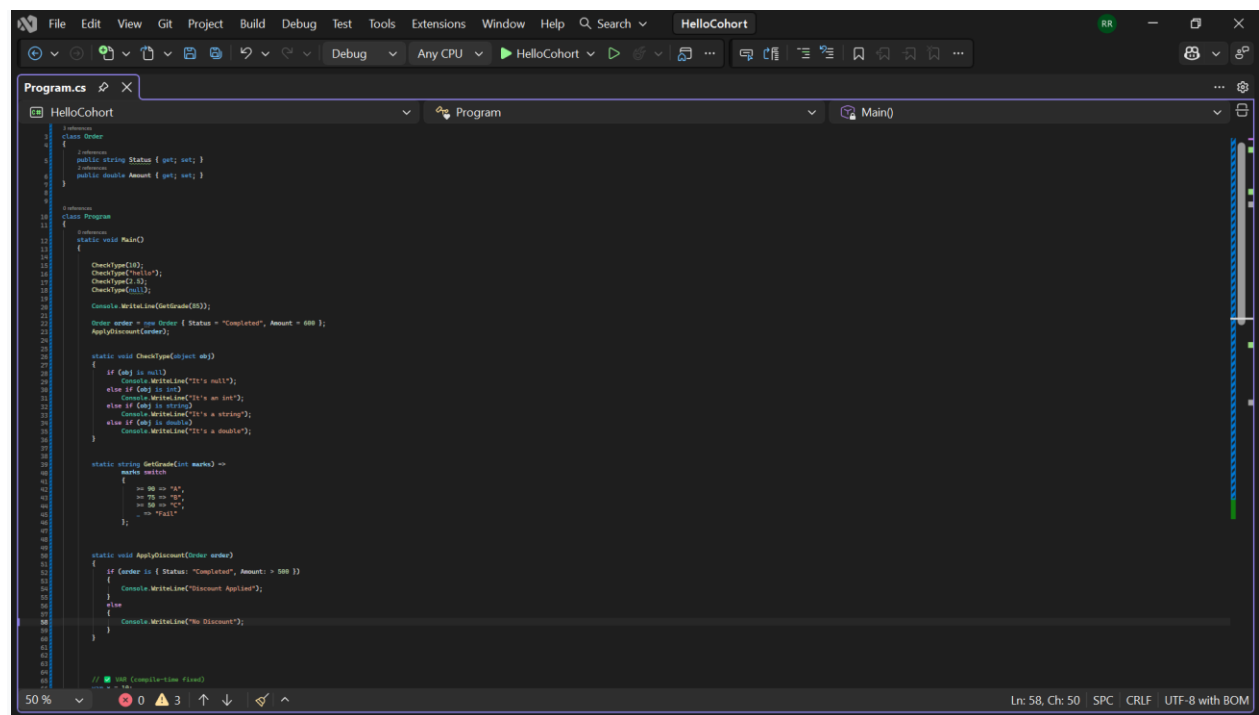
```

Task 1.15 Pattern matching: an object-param method handling int/string/double/null; a grade calculator via switch expression with relational patterns; Order {Status, Amount} discount via property patterns. ✓ Done when: each pattern path prints the correct branch.

Procedure:

Using Pattern matching, I have created an object-param method which handles int/string/double/null types. Also, created a grade calculator via switch expression with relational patterns. Built a Applydiscount function which checks the status of order using “is” keyword via property pattern.

Rithika Ravi	P575	B01	"Week 1 — C# Language & Object Foundations".
--------------	------	-----	--



```
using System;

namespace HelloCohort
{
    class Order
    {
        public string Status { get; set; }
        public double Amount { get; set; }
    }

    class Program
    {
        static void Main()
        {
            CheckType(15);
            CheckType("Hi Ravi");
            CheckType(25);
            CheckType(null);

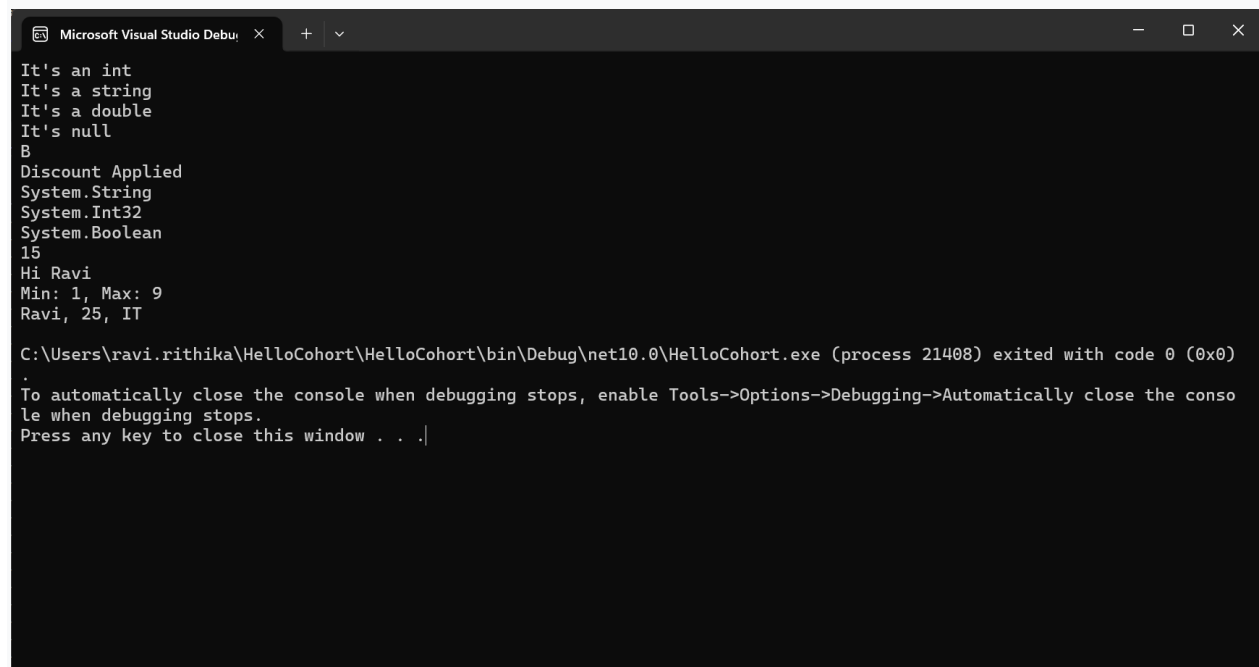
            Console.WriteLine(GetGrade(85));

            Order order = new Order { Status = "Completed", Amount = 600 };
            ApplyDiscount(order);

            static void CheckType(object obj)
            {
                if (obj is null)
                {
                    Console.WriteLine("It's null");
                }
                else if (obj is int)
                {
                    Console.WriteLine("It's an int");
                }
                else if (obj is string)
                {
                    Console.WriteLine("It's a string");
                }
                else if (obj is double)
                {
                    Console.WriteLine("It's a double");
                }
            }

            static string GetGrade(int marks) =>
            {
                switch (marks)
                {
                    case >= 80: return "A";
                    case >= 70: return "B";
                    case >= 60: return "C";
                    default: return "Fail";
                }
            }

            static void ApplyDiscount(Order order)
            {
                if (order is { Status: "Completed", Amount: > 500 })
                {
                    Console.WriteLine("Discount Applied");
                }
                else
                {
                    Console.WriteLine("No Discount");
                }
            }
        }
    }
}
```



```
Microsoft Visual Studio Debug Console

It's an int
It's a string
It's a double
It's null
B
Discount Applied
System.String
System.Int32
System.Boolean
15
Hi Ravi
Min: 1, Max: 9
Ravi, 25, IT

C:\Users\ravi.rithika\HelloCohort\HelloCohort\bin\Debug\net10.0\HelloCohort.exe (process 21408) exited with code 0 (0x0)
.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Rithika Ravi	P575	B01	"Week 1 — C# Language & Object Foundations".
--------------	------	-----	--

Task 1.16 Student class: validated properties (age 5–100), parameterized + chained constructor (: this(...)), overloaded CalculateGrade(). Add const vs readonly fields and show which can't change where. ✓ Done when: validation rejects an out-of-range age and constructor chaining works.

Procedure:

Learnt parameterized and chained constructor with const vs readonly fields. Also, learnt about method overloading using CalculateGrade().

```

91
92 //Task 1.16
93 // Constructor usage
94 Student s1 = new Student("Ravi", 25);
95 Console.WriteLine(s1.Name);
96
97 Student s2 = new Student(); // chained constructor
98 Console.WriteLine(s2.Name);
99
100 // Validation check
101 s1.Age = 2; // should print "Invalid age!"
102
103 // Method overloading
104 Console.WriteLine(s1.CalculateGrade(60)); // Pass
105 Console.WriteLine(s1.CalculateGrade(90.5)); // 8.05
106
107 // const and readonly
108 Console.WriteLine(Student.School);
109 Console.WriteLine(s1.RollNumber);
110
111 }
112
113

```

```

Program.cs
HelloCohort
Program
Main()

101 class Student
102 {
103     // Private field
104     private int age;
105
106     // Property with validation (5-100)
107     public int Age
108     {
109         get { return age; }
110         set
111         {
112             if (value < 5 || value > 100)
113                 Console.WriteLine("Invalid age!");
114             else
115                 age = value;
116         }
117     }
118
119     public string Name;
120
121     // const (cannot change anywhere)
122     public const string School = "ABC School";
123
124     // readonly (can only change in constructor)
125     public readonly int RollNumber;
126
127     // Default constructor (calls another constructor)
128     public Student() : this("Unknown", 10)
129     {
130     }
131
132     // Parameterized constructor
133     public Student(string name, int age)
134     {
135         Name = name;
136         Age = age; // validation happens here
137         RollNumber = new Random().Next(1000);
138     }
139
140     // Method overloading
141     public string CalculateGrade(int marks)
142     {
143         return marks >= 50 ? "Pass" : "Fail";
144     }
145
146     public double CalculateGrade(double marks)
147     {
148         return marks * 0.1;
149     }
150 }

```

Rithika Ravi	P575	B01	"Week 1 — C# Language & Object Foundations".
--------------	------	-----	--

```
Microsoft Visual Studio Debug Console
It's an int
It's a string
It's a double
It's null
B
Discount Applied
System.String
System.Int32
System.Boolean
15
Hi Ravi
Min: 1, Max: 9
Ravi, 25, IT
Ravi
Unknown
Invalid age!
Pass
8.05
ABC School
812

C:\Users\ravi.rithika\HelloCohort\HelloCohort\bin\Debug\net10.0\HelloCohort.exe (process 18676) exited with code 0 (0x0)
.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```